

高速路固定摄像头场景 ALPR：Windows 主线到 RK3588 板端移植、MIPI 实时识别与去模糊实验

项目目录：D:/YOLO_ALPR_Project

整理日期：2026-06-27

一句话结论

Windows Top-K 主线已经迁移到 RK3588 的视频文件识别链路，并形成了速度模式与等价诊断模式两套命令；MIPI 手机图片调试已能直接扫到车牌，但它是临时调试链路，不等同于高速路真实部署；去模糊模型在静态指标和视觉锐度上有效，但对 HyperLPR3 OCR 的收益尚不稳定，暂不建议直接进入板端实时主链路。

模块	当前状态	演示价值	主要风险
Windows 主线	Top-K、OCR 投票、锁定后显示逻辑已稳定作为基准	可展示算法从单帧到多帧投票的演进	主脚本 alpr_topk_capture.py 不应随意污染
RK 视频识别	可跑完整 144.mp4；i3 + rk_adaptive 历史完整视频约 11.72 FPS	可展示板端部署结果和浏览器预览	严格 i1 等价模式约 4.97 FPS，速度明显下降
MIPI 实时识别	颜色已通过 UYVY + gray-world 临时修正；手机图片可 direct plate scan	可现场展示摄像头画面和实时识别	手机屏幕里的整车不容易被车辆检测模型识别
Deblur	ONNX 模型已训练并可静态测试；PSNR/SSIM 提升明显	可展示 before/after 和 OCR A/B	视觉更锐不必然提升 OCR，板端实时接入成本高

1. 项目目标与当前边界

最终目标是在高速路固定摄像头场景下，尽可能接近原视频速度完成中文车牌识别。系统应在车辆进入有效识别区域时完成车牌检测与 OCR，得到足够稳定的投票结果后锁定文本；锁定后不再对该 track 重复做 OBB/OCR，只保留车辆跟踪和车牌号显示。

- Windows 端用于算法验证、A/B 试验和去模糊离线评估。
- RK3588 端用于最终实时部署，包括视频文件回放和 MIPI 摄像头输入。
- MIPI 手机图片识别是调试摄像头和实时流能力的辅助场景，不应直接等同于高速路真实车辆识别。
- 去模糊当前作为离线复核和后台实验分支，不进入板端实时主链路。

当前边界

现在主线应保持两份脚本：rk3588_topk_capture.py 保留为 Git 提交版/视频部署主线；rk3588_topk_capture_mipi_debug.py 保留 MIPI 手机图片调试直扫能力。这样不会再因为 MIPI 调试污染视频文件识别。

2. 从 Windows 到 RK3588 的移植过程

2.1 Windows 主线如何形成

- 早期 TinyUNet / 图像增强路线对论文数据集有效，但对真实高速路视频 crop 泛化不足。
- 早期 alpr_sahi_snapshot2.py 可截取车辆和车牌，但单帧 fallback 经常模糊，无法保证 OCR 稳定。
- 主线转向 Top-K：同一车辆跨多帧收集多个 plate crop，按面积、清晰度、曝光、对比度、OBB 置信度排序。
- OCR 从 PaddleOCR、plate-rec 系列逐步转向 HyperLPR3；HyperLPR3 对中文车牌整体最可靠，但仍需要投票抑制噪声。
- 最终形成 alpr_topk_capture.py：车辆检测、轻量 tracker、OBB、透视矫正、Top-K 保存、OCR 投票、锁定后显示。

2.2 移植到 RK 的关键改造

改造点	Windows 端	RK3588 端实现	遇到的问题

推理框架	Ultralytics / PyTorch / ONNXRuntime	RKNNLite 加载 vehicle.rknn 与 best_obb.rknn	RKNN 输出格式、静态 shape 警告、模型版本提示需要确认
车辆检测	YOLO yolo11n.pt	vehicle.rknn, 输入 640	每帧检测速度较慢, 需要 interval 与 adaptive
车牌 OBB	best_obb.pt	best_obb.rknn / plate_imgsz 640	命中次数少时 OCR 票数不足
OCR	HyperLPR3	板端 Python HyperLPR3	中文输出在部分日志/JSON 中会出现编码显示问题
跟踪	轻量 IoU tracker	IoU + center/scale + locked reassociation	断轨会导致锁定文字只闪一下或转移困难
实时显示	OpenCV window	/tmp/frame.jpg + stream.py	JPEG 发布会带来小幅 CPU 开销

3. 板端视频文件识别现状

板端现有视频为 /root/deploy/144.mp4, 与 Windows 端 D:/YOLO_ALPR_Project/测试图/14.mp4 内容相同。当前应区分两类运行模式：严格等价诊断模式和板端速度/效果平衡模式。

模式	核心参数	用途	观察到的 FPS/现象
Windows 等价诊断	vehicle-detect-interval=1, vehicle_conf=0.45, min_process_vehicle_conf=0.45	定位差异来源, 不追求速度	1000 帧约 4.97 FPS; 低阈值带来更多后续处理
RK 速度平衡	vehicle-detect-interval=3, --rk-adaptive, publish_interval=2	接近实时演示和完整视频测试	完整视频 11030 帧平均约 11.72 FPS; 能锁定多辆车
no-OCR 诊断	ocr-engine none / lock-on-plate-detect	证明检测后跟踪显示链路可行	约 12 FPS, 但只能显示 PLATE, 不是最终目标
MIPI 手机调试	mipi-plate-scan-only / direct plate scan	调摄像头和手机图片展示	约 6-7 FPS; 非高速路真实视频主线

为什么 4.7 FPS 和 11 FPS 都是“正常”的

4.7 FPS 来自每帧车辆检测的 Windows 等价诊断模式; 11 FPS 来自 vehicle-detect-interval=3 + rk_adaptive 的板端速度模式。二者目标不同, 不能直接比较。

3.1 历史 RK 测试结果摘要

测试	帧数	FPS	关键参数	锁定结果/结论
rk_windows_equiv_i 1	1000	4.97	interval=1	可锁定前两辆，但速度慢，适合作为等价诊断
rk_adaptive_i3	1000	12.72	interval=3, rk_adaptive	锁定 冀 JC5210、冀 B6R9F9，适合板端策略
full video run_20260626_0742 32	11030	11.72	interval=3, rk_adaptive, publish_interval=2	锁定 冀 JC5210、冀 B6R9F9、鲁 V0JU1Q、冀 CH7V97
noocr_plate_lock_ab	1000	12.03	no OCR / plate detect lock	证明锁定后跟踪显示可行，但不能作为最终识别结果

4. MIPI 摄像头识别现状

MIPI 调试经历了两个阶段：先解决浏览器是否真的看到实时摄像头画面，再定位为什么手机图片里的车辆难以进入正常车辆检测链路。

- stream.py 只是浏览器服务，读取 /tmp/frame.jpg；真正更新画面的程序必须另行运行。
- MIPI 采集使用 /dev/video22、UYVY、v4l2-ctl 后端；OpenCV V4L2 路径在板端偏慢。
- 颜色发绿/发紫主要来自 UYVY 解码、白平衡和曝光问题；已用 robust gray-world 做临时软件白平衡。
- 手机屏幕中的车辆不等同于真实车辆：车辆检测模型容易失败，但车牌 OBB 可直接在 ROI 中检测到车牌。
- 因此新增了单独调试版 rk3588_topk_capture_mipi_debug.py，支持 mipi-plate-scan-only，避免污染视频主线。

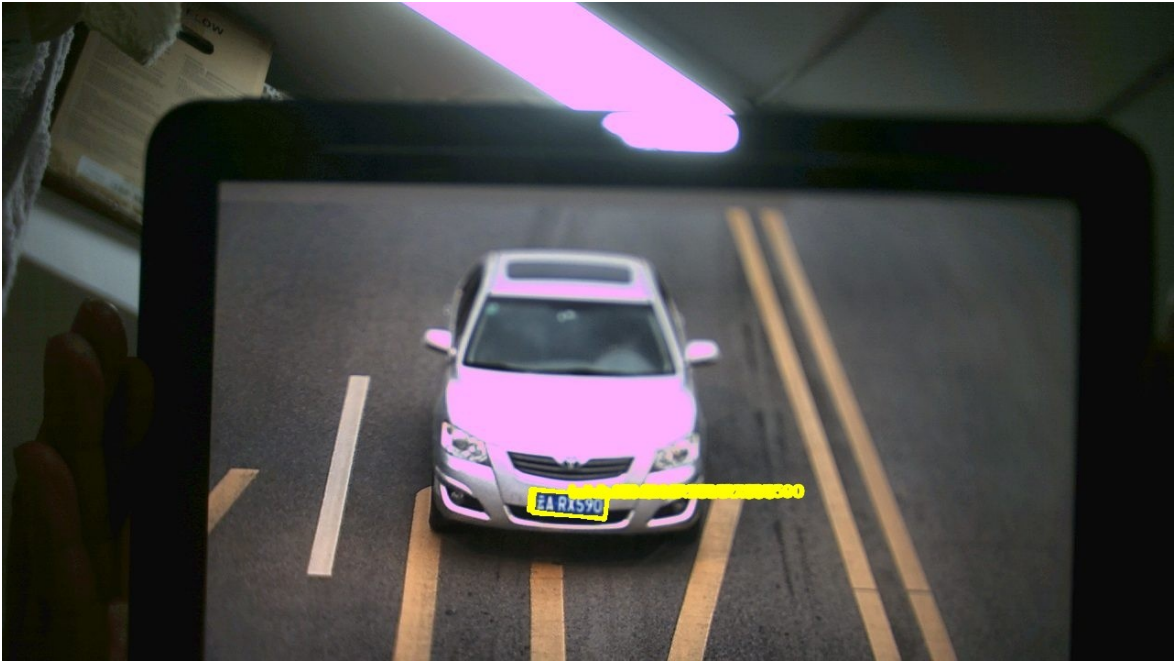


图 1：MIPI 当前帧 direct plate scan 诊断。车辆检测为空，但直接车牌 OBB + HyperLPR3 能识别手机画面中的车牌。



图 2：MIPI scan-only 轻量配置画面。该模式用于手机图片演示，不代表高速路视频主流程。

4.1 MIPI 遇到的困难与尝试

问题	现象	尝试	当前判断
----	----	----	------

浏览器画面不是实时	stream.py 只服务旧 /tmp/frame.jpg	写 mipi_preview_writer.py 持续更新帧	需要区分采集程序和浏览器服务
画面发绿/偏暗	室内背光、手机屏幕反光、白平衡漂移	UYVY 解码、gray-world、low-light gamma/CLAHE	可临时改善，但最终应确认 ISP/IQ/曝光
车辆检测失败	手机屏幕里的整车没有 vehicle box	读取当前帧跑一帧探针	车检失败但车牌 OBB 可成功
旧车牌锁定不释放	切换手机图片后仍显示上一辆	MIPI 调试版增加换牌重置	已隔离在 mipi_debug 脚本中
FPS 较低	全帧 OBB+OCR 约 2 FPS	scan-only、ROI、降低识别频率	提升到约 6-7 FPS，仍属调试模式

5. 去模糊模型实验

去模糊模型 deblur_v2_e20 已完成训练，并导出 ONNX：D:/YOLO_ALPR_Project/blur models/deblur_v2_e20/plate_restore_lite_v2_e20_320x96.onnx。当前策略是先在 Windows 端离线评估，不直接接入 RK 实时主链路。

指标	输入模糊 baseline	去模糊 restored	结论
Loss	0.0837	0.0496	损失下降
PSNR	20.54	24.55	约 +4.01 dB
SSIM	0.755	0.875	结构相似度明显提升
真实 Top-K crop 锐度	-	平均倍率约 1.24x	视觉锐度通常提升
静态 HyperLPR3 A/B	19/30 plate-like	19/30 plate-like	OCR 收益不稳定，需继续分层评估

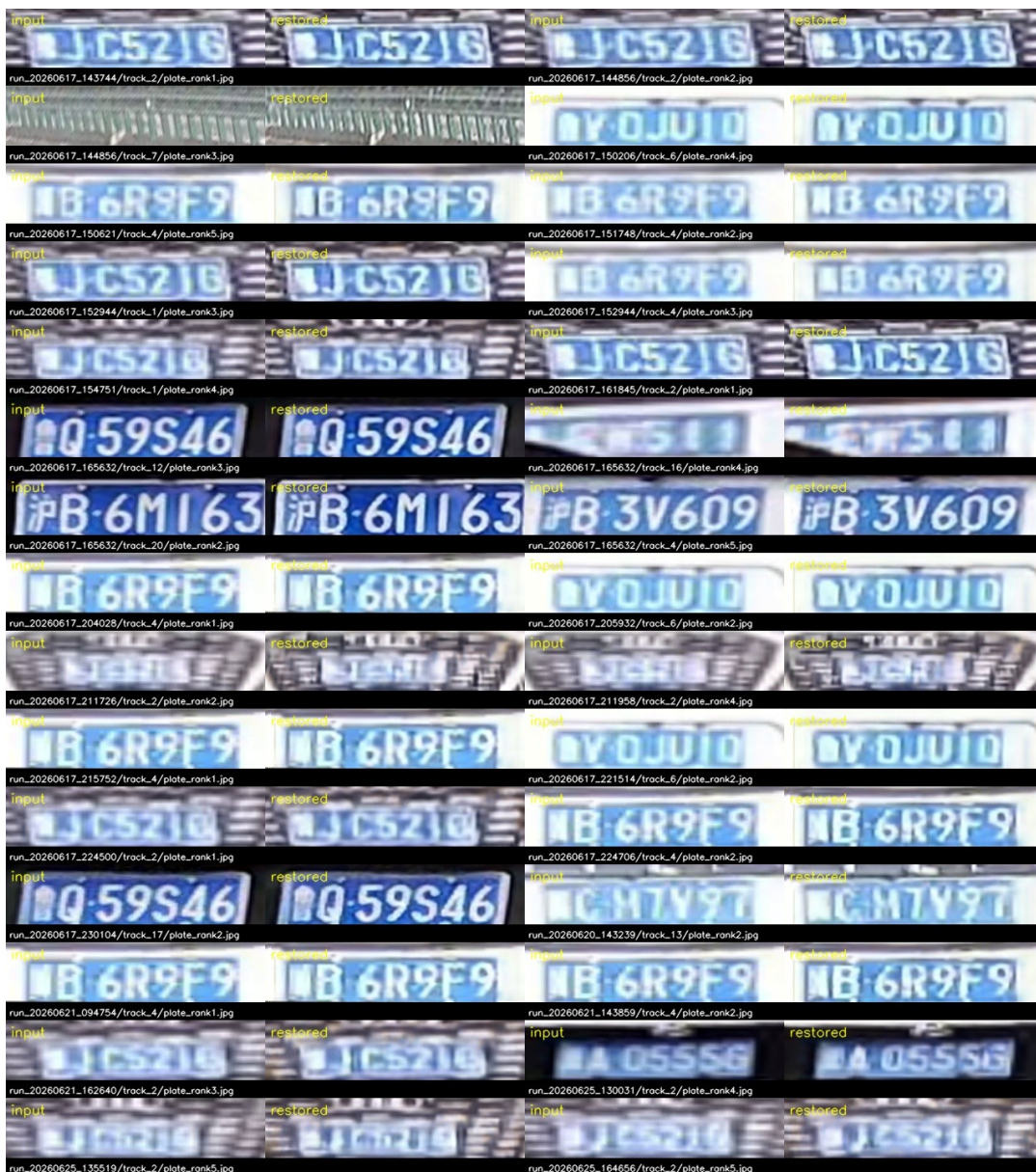


图 3: deblur_v2_e20 对真实 Top-K plate crop 的批量 before/after 预览。

去模糊当前结论

模型在合成测试集和真实 crop 的视觉锐度上有效，但 HyperLPR3 对去模糊后的文本并非总是更稳定；有些样本置信度提升，有些样本会把字符推向另一个错误结果。因此下一步应作为后台复核票源或离线 Top-K 复核层，而不是立即放入板端实时主链路。

6. 演示运行命令

6.1 板端视频速度/效果平衡模式

```
cd /root/alpr_topk_rk3588

python3 rk3588_topk_capture.py \
  --video /root/deploy/144.mp4 \
  --vehicle-model /root/deploy/vehicle.rknn \
  --plate-model /root/deploy/best_obb.rknn \
  --ocr-engine hyperlpr3 \
  --output /root/alpr_topk_rk3588/runs_rk_adaptive_video \
  --vehicle-detect-interval 3 \
  --rk-adaptive \
  --publish-frame /tmp/frame.jpg \
  --publish-interval 2 \
  --progress-interval 100
```

6.2 板端 Windows 等价诊断模式

```
cd /root/alpr_topk_rk3588

python3 rk3588_topk_capture.py \
  --video /root/deploy/144.mp4 \
  --vehicle-model /root/deploy/vehicle.rknn \
  --plate-model /root/deploy/best_obb.rknn \
  --ocr-engine hyperlpr3 \
  --output /root/alpr_topk_rk3588/runs_rk_video_equiv \
  --vehicle-detect-interval 1 \
  --vehicle-conf 0.45 \
  --min-process-vehicle-conf 0.45 \
  --min-ocr-conf 0.70 \
  --vote-window 10 \
  --vote-threshold 3 \
  --min-char-vote-ratio 0.65 \
  --publish-frame /tmp/frame.jpg \
  --publish-interval 2 \
  --progress-interval 100
```

6.3 浏览器实时流服务

```
cd /root/deploy
python3 stream.py

# Windows 浏览器打开:
http://192.168.137.168:8080
```


6.4 MIPI 手机图片调试模式

```
cd /root/alpr_topk_rk3588

python3 rk3588_topk_capture_mipi_debug.py \
--video mipi \
--camera-width 1280 \
--camera-height 720 \
--vehicle-model /root/deploy/vehicle.rknn \
--plate-model /root/deploy/best_obb.rknn \
--ocr-engine hyperlpr3 \
--output /root/alpr_topk_rk3588/runs_mipi_phone_scan \
--mipi-fourcc UYVY \
--mipi-color-mode uyvy \
--mipi-backend v4l2ctl \
--mipi-gray-world \
--rk-adaptive \
--mipi-plate-scan-only \
--mipi-plate-scan-interval 6 \
--mipi-plate-scan-conf 0.20 \
--mipi-plate-scan-roi 0.15 0.35 0.90 0.95 \
--mipi-plate-scan-reset-conf 0.85 \
--mipi-plate-scan-reset-votes 2 \
--vote-threshold 2 \
--adaptive-min-exact-votes 2 \
--adaptive-min-strong-votes 2 \
--adaptive-min-vote-frames 2 \
--min-ocr-conf 0.70 \
--publish-frame /tmp/frame.jpg \
--publish-interval 1 \
--preview-jpeg-quality 78 \
--progress-interval 30
```

6.5 Windows 端 deblur 静态图片 + HyperLPR3 测试

```
cd D:\YOLO_ALPR_Project

D:\miniconda\envs\alpr_env\python.exe .\test_deblur_hyperlpr3.py `
--input "D:\YOLO_ALPR_Project\RK3588_dev\run_20260626_074232" `
--recursive `
--output "D:\YOLO_ALPR_Project\blur models\deblur_v2_e20\static_hyperlpr3_test"

# 结果查看:
# restored\          去模糊后单图
# comparison\       原图/去模糊对照图
# results.csv        原图 OCR 与去模糊 OCR 对比
```

results.json 结构化结果

6.6 Windows demo 主流程接入 deblur 后台投票实验

```
cd D:\YOLO_ALPR_Project

D:\miniconda\envs\alpr_env\python.exe .\alpr_topk_capture_demo.py `
  --video "D:\YOLO_ALPR_Project\测试图\14.mp4" `
  --output "D:\YOLO_ALPR_Project\captures_deblur_bg_vote_full" `
  --live-ocr `
  --ocr-engine hyperlpr3 `
  --deblur-model "D:\YOLO_ALPR_Project\blur
models\deblur_v2_e20\plate_restore_lite_v2_e20_320x96.onnx" `
  --deblur-mode always `
  --progress-interval 100
```

7. 汇报时建议讲法

- 1. 先讲目标：不是单帧识别，而是固定摄像头视频中“识别一次、锁定后跟踪显示”。
- 2. 再讲 Windows 主线：从单帧 fallback 走向 Top-K、多帧投票、锁定策略。
- 3. 展示 RK 视频：先跑速度模式，让听众看到完整链路已经能在板端运行；再解释等价模式为什么慢。
- 4. 展示 MIPI：强调这是摄像头实时链路和手机图片调试，不等同于高速路模型效果；说明 direct plate scan 是为了排查车检失败。
- 5. 展示 deblur：先展示视觉 before/after，再诚实说明 OCR 收益不稳定，所以暂时放在后台复核/离线层。
- 6. 最后讲下一步：OBB 模型上板 A/B、ROI 与调度优化、MIPI ISP 正规修复、deblur 只在有收益时进入后台票源。

8. 下一步计划

优先级	任务	具体动作	验收标准
PO	稳定板端视频主线	保留 Git 版 rk3588_topk_capture.py; 每次改动跑 1000 帧以上	速度模式稳定 10-12 FPS，锁牌结果不退化

		并保存 run_summary	
P0	命令和脚本分层	主线视频脚本与 MIPI 调试脚本分离；避免调 MIPI 污染视频主线	演示命令可重复运行，行为可解释
P1	OBB e20 上板 A/B	将 best_obb_finetuned_e20.p 转 ONNX/RKNN，与 best_obb.rknn 同视频对比	plate_hits、rejected、锁定帧号、FPS 有结构化对比
P1	MIPI 正规修复	确认 media-ctl、v4l2-ctl、rkaiq_3A_server、IQ 文件与 pixel format	不依赖强软件调色也能获得稳定颜色
P1	deblur 后台复核	只对未识别或低置信 Top-K crop 做异步 deblur+OCR，再进入二级投票	证明在真实 Top-K crop 上提升锁定率且 FPS 可接受
P2	跟踪增强	必要时评估 ByteTrack/OC-SORT 或固定摄像头运动模型	减少断轨和锁定文本丢失

9. 风险与注意事项

- 不要把 MIPI 手机调试模式当作高速路真实车辆模型效果，它主要验证摄像头、流服务、颜色和车牌直扫能力。
- 不要随意改 alpr_topk_capture.py；Windows 新功能优先放 alpr_topk_capture_demo.py。
- 严格等价模式慢是预期现象；演示实时效果应使用 interval=3 + rk_adaptive。
- deblur 视觉更锐不等于 OCR 更准；必须用 results.csv 的 OCR A/B 结果说话。
- 板端测试不要只看前几百帧，至少跑到 frame=1000，完整视频结果更有说服力。
- 远程 SSH/SCP 涉及网络访问；Codex 执行时需要提升权限。

附录 A：环境与路径

对象	路径 / 地址	说明
RK3588 SSH	root@192.168.137.168	板端运行和测试

Ubuntu 开发机	apple@192.168.217.128	Ubuntu 22.04.5 LTS
板端脚本目录	/root/alpr_topk_rk3588	RK 脚本、runs 输出
板端部署目录	/root/deploy	视频、模型、stream.py
板端测试视频	/root/deploy/144.mp4	与 Windows 14.mp4 内容相同
Windows 视频	D:/YOLO_ALPR_Project/测试图/14.mp4	Windows demo 输入
实时浏览器	http://192.168.137.168:8080	stream.py 读取 /tmp/frame.jpg
deblur 模型	D:/YOLO_ALPR_Project/blur models/deblur_v2_e20	ONNX、对照图、metrics

附录 B：当前文件状态建议

截至本报告整理时，建议保持以下分工：

- rk3588_topk_capture.py：保留 Git 提交版，作为板端视频文件识别主线。
- rk3588_topk_capture_mipi_debug.py：保留 MIPI 手机图片识别、直扫 ROI、换牌重置等调试能力。
- alpr_topk_capture.py：Windows 基准主线，原则上不污染。
- alpr_topk_capture_demo.py：Windows 实验分支，可用于 deblur 后台投票实验。
- test_deblur_hyperlpr3.py：静态图片去模糊 + HyperLPR3 OCR 对比工具。